

Introdução à Ciência da Computação com Python I

Coleções e integração

REALIZAR APÓS Módulo 9 / Semana 9

Módulos 7 a 9 | 120 minutos | 100 pontos

Curso principal: Introdução à Ciência da Computação com Python Parte 1 |

<https://www.coursera.org/learn/ciencia-computacao-python-conceitos>

Conteúdo verificável: Repetições encaixadas; listas; for; manipulação; objetos na memória; estilo; integração

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Listas e memória

Execute `a=[1, 2]; b=a; c=a[:]; b.append(3); c[0]=9`. (a) Dê `a`, `b`, `c`. (b) Desenhe referências e objetos. (c) Explique a diferença entre cópia da lista e compartilhamento.

Desenvolvimento / justificativa / esquema:

Implementação e teste

Implemente `remove_repetidos(seq)`, preservando a ordem da primeira ocorrência, sem alterar `seq` e sem usar `set`. (a) Dê solução. (b) Teste lista vazia e `[3, 1, 3, 2, 1]`. (c) Explique como garante ausência de repetição.

Desenvolvimento / justificativa / esquema:

Repetições encaixadas

Implemente `pares_com_soma(a, b, alvo)`, devolvendo pares de valores um de `a` e outro de `b` que somam `alvo`. (a) Preserve as ocorrências repetidas. (b) Teste `a=[1, 2]`, `b=[2, 3]`, `alvo=4`. (c) Explique por que um único laço não examina automaticamente todas as combinações.

Desenvolvimento / justificativa / esquema:

Programa de síntese

Implemente `resumo(valores)` que devolve `[quantidade, soma, menor, maior]`, usando funções e laços. (a) Defina resultado para lista vazia com `None` nos extremos. (b) Evite inicializar `menor` com 0. (c) Teste positivos, negativos e repetidos e explique a escolha de estado inicial.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Listas e memória (15 pontos)

a e b são [1, 2, 3]; c é [9, 2]. a, b referenciam a mesma lista; a fatia cria outra lista de elementos numéricos. append por b altera o objeto visto por a. Mudar c não altera a. A cópia por fatia é superficial: se elementos fossem listas mutáveis, poderiam permanecer compartilhados; não afirmar cópia profunda universal.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Implementação e teste (25 pontos)

```
def remove_repetidos(seq):
    resultado = []
    for x in seq:
        if x not in resultado:
            resultado.append(x)
    return resultado
```

Resultados[] e [3, 1, 2]. Um elemento só é adicionado quando não aparece no resultado; a iteração segue a ordem original. Verificar que seq permanece igual após a chamada. Não é exigida análise assintótica formal nesta parte do curso.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Repetições encaixadas (25 pontos)

```
def pares_com_soma(a, b, alvo):
    pares = []
    for x in a:
        for y in b:
            if x + y == alvo:
                pares.append([x, y])
    return pares
```

Resultado

$$\begin{bmatrix} 1 & 3 \\ 2 & 2 \end{bmatrix}$$

. Para cada elemento de a percorre toda b; um laço pareando posições omitiria combinações. Testar entrada vazia e [2, 2] com b=[2], preservando dois pares iguais porque correspondem a ocorrências distintas.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Programa de síntese (35 pontos)

```
def resumo(valores):
    if len(valores) == 0:
        return [0, 0, None, None]
    menor = maior = valores[0]
    soma = 0
    for x in valores:
        soma += x
        if x < menor: menor = x
        if x > maior: maior = x
    return [len(valores), soma, menor, maior]
```

Exemplo[2, 5, 2]→[3, 9, 2, 5]; [-3, -1]→[2, -4, -3, -1]. Inicialização pelo primeiro valor impede que um extremo fictício contamine o resultado. Função não altera a lista.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.